最終課題説明資料：差動二輪ロボットのカメラ認識連動型自動走行システム

ROS 2 (Humble) / Ignition
Gazeboシミュレーション環境における画像認識とトピック制御の統合実装

講座名	Physical AI 応用編1 知能ロボティクス講座 2026 Summer	提出日	2026年11月
課題タイトル	AI×ロボティクスによる創造的タスクの実現	アカウント名	omnicampusアカウント名
開発環境	Windows + WSL2 + Docker + VSCode (noVNC)	主要技術	ROS 2 Humble / Ignition Gazebo / Python (OpenCV)

1. 課題概要と目的

本取り組みは、Physical AI 応用1講座の最終課題「AI×ロボティクスによる創造的タスクの実現」に基づき、Ignition Gazebo物理シミュレータ上の差動二輪ロボット（demoRobot）とROS 2（Humble）通信基盤を連携させ、カメラセンサからの画像認識フィードバックに基づき目標物体への自動追従・アプローチを行うシステムを構築・検証したものである。

従来のキーボードによる手動遠隔操作（テレオペ）から脱却し、カメラ画像トピックを受信して自律的に移動速度指令（/cmd_vel）を発行するAI制御ノードをPythonで実装することで、センサ入力・AI推論・アクチュエータ制御が完結する完全自動パイプラインを実現した。

> 【本タスクの到達目標】
> ① 差動二輪ロボットのSDF記述にカメラセンサ機能（<sensor type="camera">）を適切に追加・組み込みを行う。
> ② ros_ign_bridge を介して、GazeboとROS 2間で制御トピック（/cmd_vel）および画像トピック（/camera）を双方向ブリッジする。
> ③ カメラ画像内の目標物体（特定色・認識ターゲット）の位置座標を算出し、それに応じた速度指令（linear.x / angular.z）を動的生成する制御ノードを統合する。

2. システム構成と開発・実行環境

ホストPC（Windows 11）上のWSL2環境でDockerコンテナを起動し、noVNC（http://localhost:6080/）によるWebデスクトップ環境で開発を行った。データ永続化のため、compose.yaml にホスト側カレントディレクトリとコンテナ内/workspace を結ぶバインドマウント設定を追加している。

コンポーネント	採用技術・設定内容	役割・目的
ベース環境	Windows 11 + WSL2 + Docker Desktop	ホスト側の開発・コンテナ実行基盤
Dockerコンテナ	airobotbook/ros2-desktop-ai-robot-book-humble	ROS 2 Humble / Gazebo / Python環境の統一
永続化設定	volumes: - .:/workspace (compose.yaml)	作業スクリプト・設定ファイルのホスト保存
シミュレータ	Ignition Gazebo (Gazebo Sim) / world.sdf	差動二輪ロボットおよびターゲット物体の物理シミュレーション
通信ブリッジ	ros_ign_bridge parameter_bridge	ROS 2トピックとGazeboトピックの相互メッセージ変換
AI制御ノード	Python 3 (rclpy, cv_bridge, OpenCV)	カメラ画像受信、画像処理解析、移動指令生成

3. 独自の工夫点と技術的実装要点

【工夫点 1】SDFモデルへのカメラセンサ機能組み込み

既存の差動二輪モデル（demoRobot）には車輪および車体のリンクのみが定義され、光学センサが欠落していた。そこで、world.sdf 内のメイン車体リンク（chassis）直下に <sensor type="camera"> 記述を追加するとともに、<world> タグ配下に ignition-gazebo-sensors-system プラグインを追加配置することで、シミュレーション内でのリアルタイムカメラ画像生成（/camera）を有効化した。

【工夫点 2】ROS 2 - Gazebo 通信パイプラインの構築

速度指令 /cmd_vel（geometry_msgs/msg/Twist）とカメラ画像 /camera（sensor_msgs/msg/Image）の両トピックを同時にROS 2へ転送するため、以下のパラメータブリッジコマンドを構成・実行した。

```
$ ros2 run ros_ign_bridge parameter_bridge \
/cmd_vel@geometry_msgs/msg/Twist]ignition.msgs.Twist \
/camera@sensor_msgs/msg/Image[ignition.msgs.Image
```

【工夫点 3】画像認識連携による自律追従アルゴリズムノード

rclpy と cv_bridge を活用し、/camera トピックをサブスクライブするPythonノード（camera_tracker.py）を開発した。受信した画像内のターゲット位置偏差（X座標偏差）を算出し、目標が左にある場合は左旋回（angular.z > 0）、右にある場合は右旋回（angular.z < 0）、中央付近では前進（linear.x > 0）を行うフィードバック制御ロジックを組み込んだ。

4. 実験結果および考察

【動作検証結果】
Ignition Gazebo上でワールド（world.sdf）を起動し、ブリッジノードおよび開発した認識制御ノードを実行した結果、ロボットはカメラ視野内に捉えたターゲット物体に向かって自動的に旋回・前進を行い、適正距離までアプローチして安全に追従動作を継続することが確認された。

【考察とRoboOps的知見】
1. 環境初期化とデータ永続化：Dockerコンテナ環境特有の問題（コンテナ再作成時のファイル消滅）に対し、WSL2上のボリュームマウント設定およびdocker compose の活用により安定した開発サイクルを確立した。
2. トピック通信のデバッグ：新しいターミナル起動時に source /opt/ros/humble/setup.bash およびワークスペースのセットアップを忘れるとノード未検出エラーとなるため、環境変数設定の確実な実行と通信一覧の試算・確認の重要性を実感した。
3. 将来的な拡張性：本構成は軽量な認識アルゴリズムに基づいているが、学習済みYOLOv8やLeRobot等による深層学習推論モデルへのノード単位での置き換えが容易であり、より複雑なマルチクラス物体認識・自動回避タスクへスムーズに発展可能な高い柔軟性を有している。

5. 最終提出物一覧

提出物項目	提出形式 / ファイル名	概要・記入先
① レポート資料 (PDF)	説明資料_PAI最終課題_omnicampusアカウント名.pdf	Omnicampusフォームより直接アップロード提出
② ソースコード	GitHub Publicリポジトリ	OmnicampusフォームにPublicリポジトリURLを記載
③ 実証デモ動画	YouTube（限定公開 / 1〜3分）	Omnicampusフォームに限定公開動画URLを記載